Expert Systems

Lecture 8: Artificial neural networks-single layer

Dr. Mohammed A. Altahrawi

University College of Applied Sciences

February 24, 2026

Outlines

1. Neural Network Definition
2. ANN as simple computing element
3. The Perceptron
4. Perceptron Learning

Outlines

1. Neural Network Definition
2. ANN as simple computing element
3. The Perceptron
4. Perceptron Learning

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

What is a Neural Network?

A neural network is a model of reasoning inspired by the human brain.

- The brain consists of interconnected nerve cells called **neurons**.
- The human brain has nearly 10 billion neurons.
- Around 60 trillion connections (synapses) link them.

Important Observation

By using many neurons simultaneously, the brain performs tasks faster than even the most powerful computers.

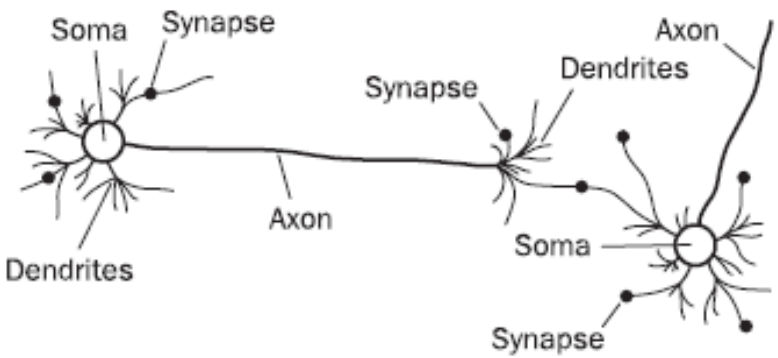
Structure of a Biological Neuron

Each neuron has a simple structure:

- Cell body (**soma**)
- Dendrites (input fibres)
- Axon (output fibre)

Signal Flow

- Signals travel through dendrites.
- If a threshold is reached, an action potential is triggered.
- The electrical pulse travels through the axon.
- It affects other neurons via synapses.



Brain vs Artificial Neural Networks

The brain is:

- Highly complex
- Non-linear
- Parallel information-processing system

Information is processed globally rather than locally.

Core Insight

The ability of biological neural networks to learn naturally inspired the development of artificial neural networks in computers.

ANNs vs the Human Brain

Although a present-day Artificial Neural Network (ANN) resembles the human brain much like a paper plane resembles a supersonic jet, it represents a significant step forward.

ANNs are capable of **learning**:

- They use experience to improve performance.
- They generalize from observed samples.
- They adapt to new, unseen data.

Core Capability

When exposed to sufficient data, ANNs can generalize to situations they have never encountered before.

Applications of ANNs

Artificial Neural Networks can:

- Recognize handwritten characters
- Identify words in human speech
- Detect explosives at airports
- Discover hidden patterns in large datasets

Real-World Example: Fraud Detection

Chase Manhattan Bank used a neural network to analyze stolen credit card transactions.
The network discovered that the most suspicious sales were:

- Women's shoes
- Priced between $40 and $80

This pattern had not been clearly recognized by human experts.

Why This Matters

- ANNs can detect patterns humans may overlook.
- They scale to massive datasets.
- They improve continuously with more data.

Key Insight

Artificial Neural Networks extend human analytical capabilities by uncovering complex and hidden relationships within data.

How Do Artificial Neural Networks Model the Brain?

An Artificial Neural Network (ANN) consists of a number of very simple and highly interconnected processors, also called **neurons**, which are analogous to biological neurons in the brain.

- Neurons are connected by **weighted links**.
- Each neuron receives multiple **input signals**.
- A neuron produces only **one output signal**.
- The output is transmitted through an outgoing connection (similar to a biological axon).
- The outgoing connection may branch, transmitting the same signal to multiple neurons.

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

Signal Transmission in ANN

- The signal is **not divided** among outgoing branches.
- Outgoing branches terminate at the incoming connections of other neurons.
- This structure forms a highly interconnected computational network.

Conclusion:
Artificial neural networks mimic the structural behavior of biological neural systems through interconnected processing units and weighted signal transmission.

How Does an Artificial Neural Network Learn?

Core Idea

Neurons are connected by links, and each link has a **numerical weight**. Weights represent the strength (importance) of each input and act as the **long-term memory** of the network.

Learning Mechanism

A neural network **learns** through repeated adjustments of these weights.

Analogy: Biological vs Artificial Networks

Structural Correspondence

Biological Network	Artificial Network
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight

How Are the Weights Adjusted?

Network Structure

A typical ANN is organized into a hierarchy of layers:

- Input Layer
- Hidden Layer(s)
- Output Layer

Weights are modified to align the network's output with the desired environment behaviour.

Important Concept

Each neuron computes its **activation level** using:

$$\text{Activation} = f\left(\sum w_i x_i\right)$$

where x_i are inputs and w_i are weights.

Designing and Training an ANN

Design Steps

1. Decide the number of neurons.
2. Define the network architecture.
3. Choose a learning algorithm.

Training Phase

- Initialize weights.
- Provide training examples.
- Update weights iteratively.

Outlines

1. Neural Network Definition
2. ANN as simple computing element
3. The Perceptron
4. Perceptron Learning

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

The Neuron as a Simple Computing Element

Basic Function

A neuron:

- Receives multiple **input signals**
- Computes an **activation level**
- Sends a single **output signal**

Important

The output may represent:

- A final solution, or
- An input to other neurons

Mathematical Model of a Neuron

Computation

For inputs $x_1, x_2, \ldots, x_n$ and weights $w_1, w_2, \ldots, w_n$:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

- w_i: weights
- b: bias
- $f(\cdot)$: activation function

Diagram of a Neuron

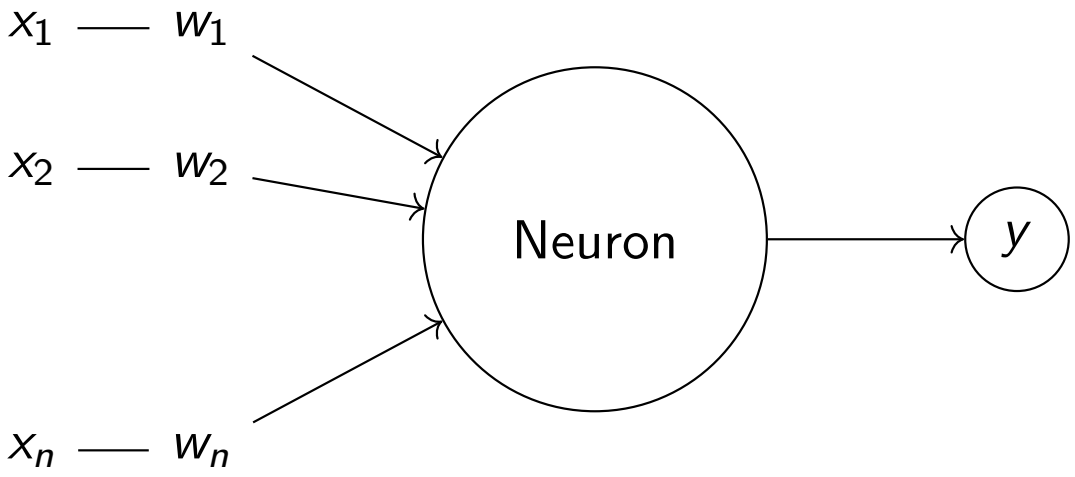


Figure: A neuron receives weighted inputs and produces a single output.

How Does the Neuron Determine Its Output?

Historical Foundation (1943)

McCulloch and Pitts proposed a simple neuron model that is still the basis of most ANNs.

Weighted Sum Computation

The neuron computes the net input:

$$X = \sum_{i=1}^{n} x_i w_i$$

It compares X with a threshold value θ.

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

Sign Activation Function

Decision Rule

$$Y = \begin{cases} +1 & \text{if } X \geq \theta \\ -1 & \text{if } X < \theta \end{cases}$$

Interpretation

- x_i: input value
- w_i: weight of input i
- n: number of inputs
- X: net weighted input
- Y: neuron output

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

Compact Representation

Sign Function Form

The neuron output can be written as:

$$Y = \text{sign}\left(\sum_{i=1}^{n} x_i w_i - \theta\right)$$

Key Idea

The sign activation function is also called a **hard-limit function** and is widely used in:

- Classification
- Decision making
- Pattern recognition

Is Sign the Only Activation Function?

Other Common Activation Functions

Although many have been proposed, common practical ones include:

- Step Function
- Sign Function
- Linear Function
- Sigmoid Function

Note

Step and sign functions are often called **hard-limit functions**.

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

Activation Functions

Sigmoid Function

The sigmoid function transforms the input, which can have any value between $-\infty$ and $+\infty$, into a reasonable value in the range between 0 and 1.

Neurons with this function are commonly used in back-propagation networks.

Linear Activation Function

The linear activation function provides an output equal to the neuron's weighted input.

Neurons with the linear function are often used for linear approximation problems.

Activation functions

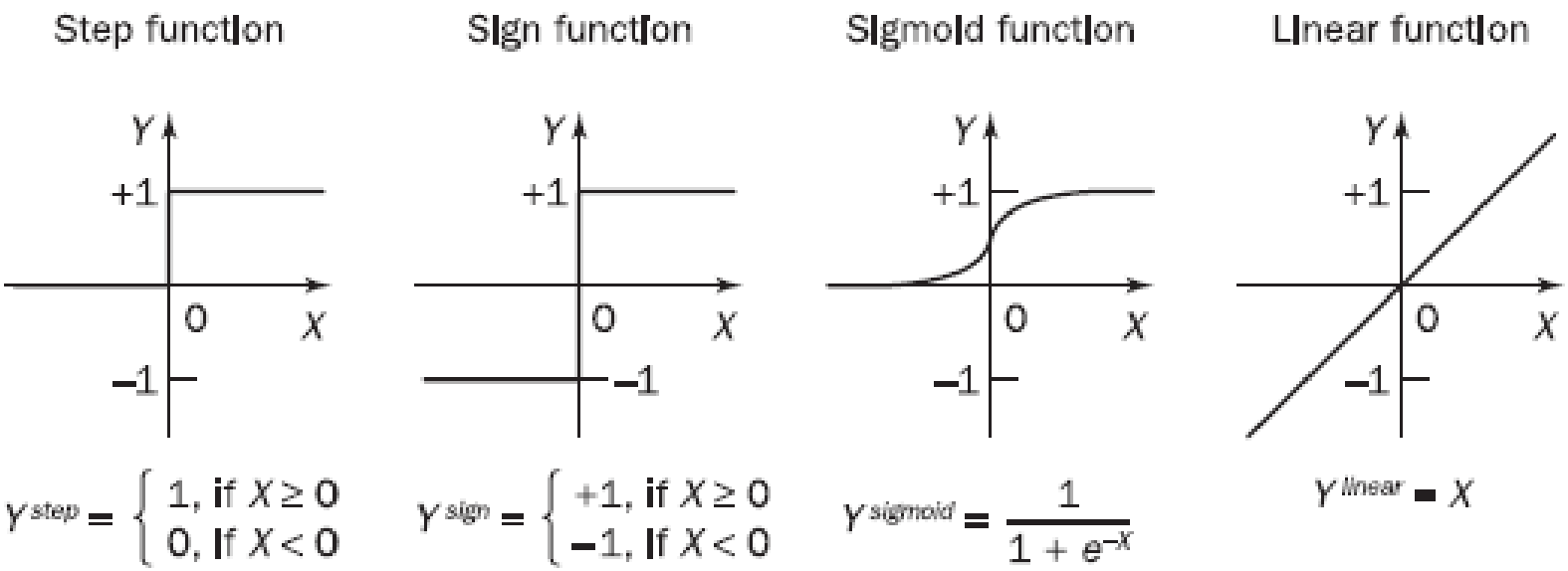


Figure: Activation functions of a neuron

Can a Single Neuron Learn a Task?

Historical Background

In 1958, Frank Rosenblatt introduced a training algorithm that provided the first procedure for training a simple artificial neural network: the **Perceptron**.

What is a Perceptron?

The perceptron is the simplest form of a neural network. It consists of:

- A single neuron
- Adjustable synaptic weights
- A hard limiter (step activation function)

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

Single-Layer Two-Input Perceptron

Structure

A single-layer, two-input perceptron consists of:

- Two input signals x_1 and x_2
- Corresponding weights w_1 and w_2
- A bias term b
- A hard limit activation function

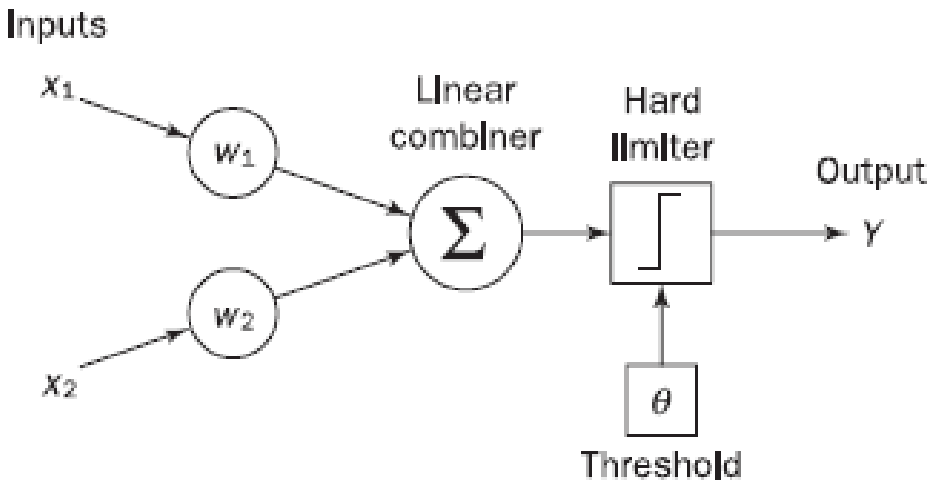
Mathematical Model

$$y = f(w_1 x_1 + w_2 x_2 + b)$$

Where $f(\cdot)$ is a hard limiter:

$$f(n) = \begin{cases} 1 & \text{if } n \geq 0 \\ 0 & \text{if } n < 0 \end{cases}$$

Single-layer two-input perceptron



Outlines

1. Neural Network Definition
2. ANN as simple computing element
3. The Perceptron
4. Perceptron Learning

The Perceptron

Rosenblatt's Perceptron Model

The operation of Rosenblatt's perceptron is based on the McCulloch and Pitts neuron model.

The model consists of:

- A linear combiner of inputs
- A hard limiter

The weighted sum of the inputs is applied to the hard limiter, which produces:

$$\text{output} = \begin{cases} +1 & \text{if input is positive} \\ -1 & \text{if input is negative} \end{cases}$$

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

Perceptron for Classification

Purpose

The perceptron aims to classify inputs (stimuli $x_1, x_2, \ldots, x_n$) into one of two classes, A_1 or A_2.

Decision Boundary

In an elementary perceptron, the n-dimensional input space is divided by a hyperplane into two decision regions. The hyperplane is defined by a linearly separable function:

$$\sum_{i=1}^{n} x_i w_i - \theta = 0$$

For the two-input case (x_1 and x_2), the decision boundary becomes a straight line.

Two-Input Perceptron Example

Class Assignment

- Point 1, above the boundary line, belongs to class A_1
- Point 2, below the line, belongs to class A_2

Threshold Adjustment

The threshold θ can be used to shift the decision boundary in the input space.

Three-Input Perceptron

Visualization

With three inputs, the hyperplane can still be visualized in 3D space. The separating plane is defined by:

$$x_1 w_1 + x_2 w_2 + x_3 w_3 - \theta = 0$$

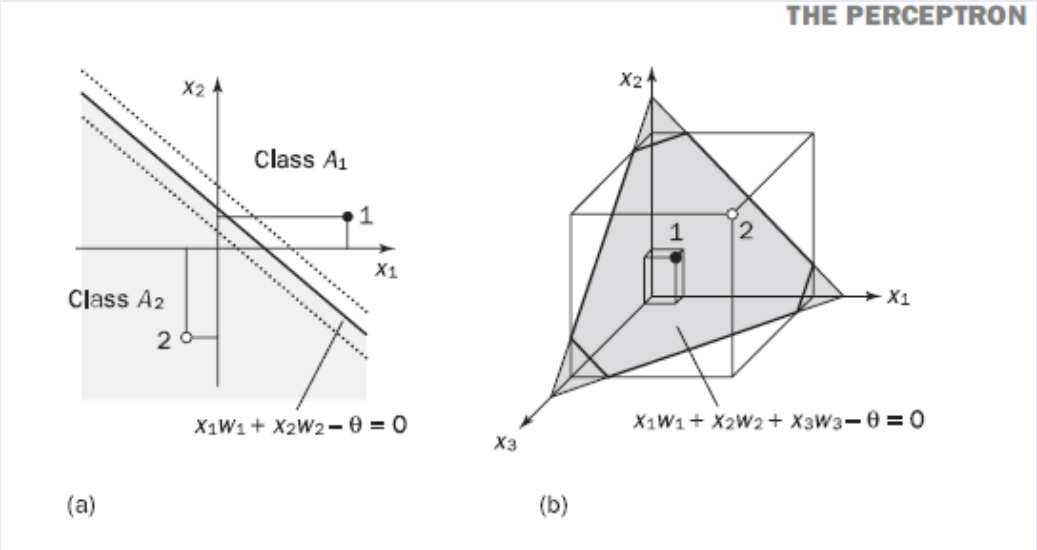


Figure: Linear separability in the perceptrons: (a) two-input perceptron;(b) three-input perceptron

Outlines

1. Neural Network Definition
2. ANN as simple computing element
3. The Perceptron
4. Perceptron Learning

Perceptron Learning Overview

Learning Concept

The perceptron learns classification tasks by adjusting its weights to reduce the difference between actual output Y and desired output Y_d.

Initial weights are usually randomly assigned in the range $[-0.5, 0.5]$, and then updated iteratively based on training examples.

Error Calculation

For iteration p, the error is:

$$e(p) = Y_d(p) - Y(p), \quad p = 1, 2, 3, \ldots$$

- If $e(p) > 0$, increase $Y(p)$
- If $e(p) < 0$, decrease $Y(p)$

Perceptron Weight Update Rule

Intuition

Each input $x_i(p)$ contributes $x_i(p)w_i(p)$ to the total input $X(p)$.

- If $x_i(p) > 0$, increasing $w_i(p)$ increases $Y(p)$
- If $x_i(p) < 0$, increasing $w_i(p)$ decreases $Y(p)$

Learning Rule (Rosenblatt, 1960)

The perceptron weight update formula:

$$w_i(p+1) = w_i(p) + \eta\, x_i(p)\, e(p)$$

- η is the learning rate $(0 < \eta < 1)$
- This simple rule allows iterative training for classification

Summary of Perceptron Learning

Training Algorithm Overview

1. Initialize weights randomly
2. Present training examples iteratively
3. Compute output $Y(p)$ and error $e(p)$
4. Update weights using $w_i(p+1) = w_i(p) + \eta x_i(p)e(p)$
5. Repeat until outputs match desired classes or max iterations reached

Perceptron Training Algorithm: Step-by-Step

Step 1: Initialization

Set initial weights $w_1, w_2, \ldots, w_n$ and threshold θ to random numbers in the range $[-0.5, 0.5]$.

Step 2: Activation

Activate the perceptron by applying inputs $x_1(p), x_2(p), \ldots, x_n(p)$ and the desired output $Y_d(p)$.
Calculate the actual output:

$$Y(p) = \text{step}\left(\sum_{i=1}^{n} x_i(p) w_i(p) - \theta\right)$$

where step$(\cdot)$ is the step activation function.

Cont.,

Step 3: Weight Training

Update the weights using:

$$w_i(p+1) = w_i(p) + \Delta w_i(p)$$

Weight correction is computed by the delta rule:

$$\Delta w_i(p) = \eta\, x_i(p)\, e(p)$$

where η is the learning rate and $e(p) = Y_d(p) - Y(p)$.

Step 4: Iteration

Increase p by 1, return to Step 2, and repeat until the weights converge.

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

Training a Perceptron for Logical Operations

Goal

Can we train a perceptron to perform basic logical operations such as AND, OR, or Exclusive-OR (XOR)?

Truth Tables for Two Inputs

The perceptron must classify input patterns according to the following truth tables:

x_1	x_2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

Training Example: AND Operation

Process

1. After initialization, activate the perceptron with the sequence of four input patterns (an epoch).
2. Update weights after each activation.
3. Repeat the process until all weights converge to a uniform set of values.

Results

The final weights and outputs after convergence are typically summarized in a table (like Table 6.3 in the text).

Table 6.3

Epoch	Inputs x_1	Inputs x_2	Desired output Y_d	Initial weights w_1	Initial weights w_2	Actual output Y	Error e	Final weights w_1	Final weights w_2
1	0	0	0	0.3	−0.1	0	0	0.3	−0.1
	0	1	0	0.3	−0.1	0	0	0.3	−0.1
	1	0	0	0.3	−0.1	1	−1	0.2	−0.1
	1	1	1	0.2	−0.1	0	1	0.3	0.0
2	0	0	0	0.3	0.0	0	0	0.3	0.0
	0	1	0	0.3	0.0	0	0	0.3	0.0
	1	0	0	0.3	0.0	1	−1	0.2	0.0
	1	1	1	0.2	0.0	1	0	0.2	0.0
3	0	0	0	0.2	0.0	0	0	0.2	0.0
	0	1	0	0.2	0.0	0	0	0.2	0.0
	1	0	0	0.2	0.0	1	−1	0.1	0.0
	1	1	1	0.1	0.0	0	1	0.2	0.1
4	0	0	0	0.2	0.1	0	0	0.2	0.1
	0	1	0	0.2	0.1	0	0	0.2	0.1
	1	0	0	0.2	0.1	1	−1	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1
5	0	0	0	0.1	0.1	0	0	0.1	0.1
	0	1	0	0.1	0.1	0	0	0.1	0.1
	1	0	0	0.1	0.1	0	0	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

Perceptron for OR and XOR Operations

OR Operation

Similar to the AND operation, a perceptron can also learn the OR operation.

- Training follows the same steps: initialization, activation, weight update, and iteration.
- The perceptron converges to weights that correctly classify all OR input patterns.

XOR Operation

A single-layer perceptron **cannot** learn the XOR operation.

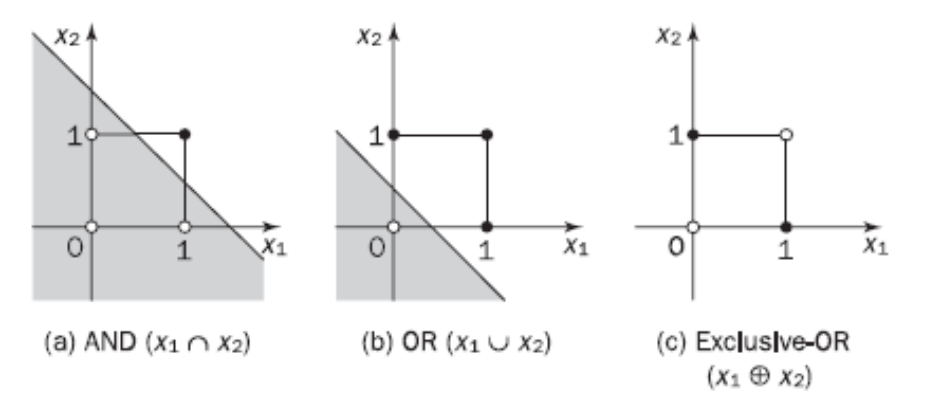
- XOR is not linearly separable.
- No single straight line (or hyperplane) can separate the 1s from the 0s in the input space.

Geometric Illustration of Logical Functions

Visualizing AND, OR, and XOR

Figure 6.7 shows the two-dimensional input space for two-input logical functions:

- Black dots represent points where the function output is 1
- White dots represent points where the output is 0



Linear Separability of Logical Functions

Observation from Figures 6.7(a)–(c) last frame

- In Figures 6.7(a) and (b), a line can separate black dots (output 1) from white dots (output 0).
- In Figure 6.7(c) (XOR), no single line can separate the points.

Conclusion

A perceptron can represent a function only if there exists a line (or hyperplane) that separates all outputs of 1 from outputs of 0. Such functions are **linearly separable**.

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

Why Single-Layer Perceptrons Learn Only Linearly Separable Functions

Perceptron Output Rule

The perceptron output Y is 1 only if the total weighted input X is greater than or equal to the threshold θ:

$$X = \sum_{i=1}^{n} x_i w_i, \quad Y = 1 \text{ if } X \geq \theta$$

This divides the input space along the boundary $X = \theta$.

Example

Example: AND Operation

For a two-input perceptron (weights and threshold from Table 6.3):

$$0.1x_1 + 0.1x_2 = 0.2 \quad \Rightarrow \quad x_1 + x_2 = 2$$

- Region below line: $x_1 + x_2 - 2 < 0$ (output 0)
- Region above line: $x_1 + x_2 - 2 > 0$ (output 1)

Limitations of Single-Layer Perceptrons

Activation Function Does Not Help

Using a sigmoidal or linear element instead of a hard limiter does not overcome the limitation:

- Single-layer perceptrons classify only linearly separable patterns.
- The decision boundary is always linear, regardless of activation function.

Computational Limitations

Minsky and Papert (1969) mathematically analyzed perceptrons:

- Cannot make global generalizations from locally learned examples.
- Cannot solve problems like XOR that are not linearly separable.

Overcoming Perceptron Limitations

Limitations Extend Beyond Single-Layer Perceptrons

- The limitations of a single-layer perceptron also apply to simple multilayer neural networks if not properly trained.
- This realization initially discouraged research on artificial neural networks (ANNs).

Coping with Non-Linearly Separable Problems

- To solve problems that are not linearly separable, we need **multilayer neural networks**.
- Advanced networks, such as **multilayer perceptrons (MLPs)** trained with the **back-propagation algorithm**, can overcome the limitations of Rosenblatt's perceptron.

الكلية الجامعية للعلوم التطبيقية
University College of Applied Sciences

THANKS